

Atelier 1 : la saisie fiable des livraisons

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Chapitre** : 1, Gérer les erreurs · **Version** : 2026-10-18

Avant de commencer

Ce que vous allez construire	Le programme de saisie des livraisons de Yonnee Express : date, quartier, poids, valeur du colis, calcul des frais, récapitulatif. Il **ne plante jamais**, quelle que soit la saisie
Durée	🟢 Débutant : 4 h · 🔵 Standard : 3 h · 🟣 Expert : 2 h (sans les défis)
Prérequis	Avoir lu le chapitre 1 ; Python 3.12 ou plus récent, VS Code, Git, un compte GitHub
Fichiers fournis	Aucun : vous partez d'un dossier vide
Rendu	L'adresse de votre dépôt GitHub `yonnee-express`, avec l'étiquette `atelier-1`

Les règles de Yonnee Express :

Règle	Valeur
Quartiers desservis et tarif de base	Médina 1000 F · Plateau 1500 F · Yoff 2000 F · Parcelles 2000 F · Pikine 2500 F
Supplément colis lourd	+ 500 F au-delà de 5 kg
Valeur d'un colis acceptée	De 500 F à 500 000 F
Date d'une livraison	Au format JJ/MM/AAAA, et jamais dans le futur

Ce que votre programme affichera à la fin, pour deux livraisons saisies avec quelques erreurs (les saisies de l'utilisateur sont après les `:`) :

```
=== Yonnee Express : saisie des livraisons ===
Nombre de livraisons à saisir (1 à 20) : trois
  « trois » n'est pas un nombre entier.
Nombre de livraisons à saisir (1 à 20) : 2
--- Livraison 1 ---
Date (JJ/MM/AAAA) : 31/02/2026
  Date invalide : utilisez le format JJ/MM/AAAA (par exemple 18/10/2026).
Date (JJ/MM/AAAA) : 15/10/2026
Quartier : Guédiawaye
  Quartier non desservi. Quartiers possibles : Médina, Plateau, Yoff, Parcelles, Pikine
Quartier : plateau
Poids (kg) : 2,5
Valeur du colis (F) : 200
  Montant refusé (le montant doit être entre 500 et 500000 F).
Valeur du colis (F) : 12 500
--- Livraison 2 ---
...
Récapitulatif :
  15/10/2026 | Plateau   |   2.5 kg | colis   12500 F | frais 1500 F
  16/10/2026 | Yoff      |   7.5 kg | colis   85000 F | frais 2500 F
Total des frais : 4000 F
```

> 📍 **Point de départ : un nouveau projet**, que vous créez à l'étape 0. **Gardez-le** : c'est le projet du fil rouge pour tout le cours.
>
> **La suite du fil rouge** : atelier 1 (la saisie) → atelier 2 (le même projet : enregistrer et recharger le registre dans des fichiers) → ateliers 3 à 6 (statistiques, rapports, gros volumes, organisation en modules) → atelier 7 (le registre réécrit avec des classes).

Comment travailler

L'atelier est découpé en **9 étapes** (0 à 8). Chaque étape suit le même plan : 🎯 **Objectif** · 🛠️ **A faire** · 🔎 **A chercher** · 💡 **A comprendre** (répondez en commentaire `#` dans le code) · ✅ **Vérification** · ➕ **Vous pouvez ajouter** (facultatif).

Trois règles pour tout l'atelier

1. **Testez chaque fonction avec de mauvaises saisies** : c'est le cœur de l'atelier. Pour chaque fonction, essayez au moins une saisie vide, une saisie en lettres et une valeur hors limites.
2. **Une étape réussie = un commit.**
3. **Aucun `except:` seul** (chapitre 1, section 3.4).

> **A propos des assistants IA** : vous pouvez demander à une IA d'**expliquer** un traceback. Ne lui faites pas écrire l'étape : au contrôle sur machine, vous serez seul.

Étape 0 : créer le projet (15 min)

🎯 **Objectif** : un dossier `yonnee-express` avec un fichier `livraisons.py`, sous Git et sur GitHub.

🛠️ **A faire**

1. Créez le dossier `yonnee-express`, ouvrez-le dans VS Code, créez `livraisons.py` avec une seule ligne : `print("=== Yonnee Express : saisie des livraisons ===")`.
2. Lancez-le dans le terminal de VS Code (`Ctrl+ù`, ou **Terminal > Nouveau terminal**) : `python livraisons.py`.
3. Créez un dépôt **vide** `yonnee-express` sur GitHub, puis :

```
git init
git add .
git commit -m "Crée le projet yonnee-express"
git branch -M main
git remote add origin https://github.com/<votre-compte>/yonnee-express.git
git push -u origin main
```

✅ **Vérification** : le titre s'affiche ; GitHub montre `livraisons.py`.

Étape 1 : les règles en constantes (10 min)

🎯 **Objectif** : toutes les règles de Yonnee Express sont écrites **une seule fois**, en haut du fichier.

🛠️ **A faire**

1. En haut de `livraisons.py`, créez le dictionnaire `TARIFS` (quartier vers tarif de base) et les constantes `SUPPLEMENT_LOURD`, `POIDS_LOURD`, `MONTANT_MIN` et `MONTANT_MAX`. Par convention, les constantes s'écrivent en MAJUSCULES.
2. Affichez provisoirement la liste des quartiers, séparés par des virgules, avec `", ".join(TARIFS)`.

💡 **A comprendre**

- Que donne `", ".join(TARIFS)` quand `TARIFS` est un dictionnaire ? Pourquoi ?
- Python accepte `500_000` pour écrire 500 000 : à quoi sert le `_` ?

✅ **Vérification** : `Médina, Plateau, Yoff, Parcelles, Pikine` s'affiche. Supprimez ensuite cet affichage provisoire.

```
git add .
git commit -m "Ajoute les règles de Yonnee Express"
```

Étape 2 : lire un entier entre deux bornes (20 min)

🎯 **Objectif** : une fonction `lire_entier(question, minimum, maximum)` qui redemande tant que la saisie n'est pas un entier dans les bornes.

🛠️ A faire

1. Écrivez la fonction (chapitre 1, section 4). Enlevez les espaces autour de la saisie avec `strip()`.
2. Utilisez-la pour demander le nombre de livraisons à saisir (1 à 20), et affichez-le.

💡 A comprendre

- Pourquoi le `continue` est-il nécessaire dans le `except` ?
- Que se passe-t-il si l'utilisateur tape `7` (avec des espaces) ? Et `7.5` ?

✅ **Vérification** : `trois`, `0`, `25` et une saisie vide sont refusés avec le bon message ; `2` est accepté.

```
git add .
git commit -m "Lit un entier entre deux bornes"
```

Étape 3 : le montant, avec `raise` (25 min)

🎯 **Objectif** : la valeur du colis est refusée hors de 500 à 500 000 F, et l'utilisateur peut taper `12 500` avec un espace.

🛠️ A faire

1. Écrivez `verifier_montant(montant)` qui **lève** une `ValueError` avec le message `le montant doit être entre 500 et 500000 F` si le montant est hors limites, et le renvoie sinon. Elle n'affiche rien et ne lit rien.
2. Écrivez `lire_montant(question)` : elle lit le texte, supprime **tous** les espaces (`replace(" ", "")`), convertit avec `int`, appelle `verifier_montant`, et redemande en affichant `Montant refusé (...)` avec le message de l'exception.

🔎 **A chercher** : chapitre 1, sections 3.2 (`as e`) et 5.1 (`raise`).

💡 A comprendre

- Un seul `except ValueError as e` suffit pour **deux** problèmes différents : lesquels ? D'où vient chacun des deux messages ?
- Pourquoi `verifier_montant` ne fait-elle pas elle-même le `print` ?

✅ **Vérification** : `200` affiche `Montant refusé (le montant doit être entre 500 et 500000 F).` ; `douze mille` affiche le message de `int()` ; `12 500` est accepté.

```
git add .
git commit -m "Lit et vérifie la valeur du colis"
```

Étape 4 : le poids, avec une virgule ou un point (15 min)

🎯 **Objectif** : `lire_poids(question)` accepte `2,5` et `2.5`, et refuse ce qui n'est pas un nombre positif.

🛠️ **A faire**

1. Remplacez la virgule par un point avant `float()`.
2. Refusez les poids nuls ou négatifs, avec un message différent de celui des saisies non numériques.

✅ **Vérification** : `deux` → « n'est pas un poids » ; `-1` et `0` → « Le poids doit être positif » ; `2,5` → 2.5.

```
git add .
git commit -m "Lit le poids du colis"
```

Étape 5 : la date, avec le module `datetime` (35 min)

🎯 **Objectif** : `lire_date(question)` accepte une date valide au format JJ/MM/AAAA, qui n'est pas dans le futur.

🛠️ **A faire**

1. En haut du fichier : `from datetime import date, datetime`.
2. Convertissez le texte avec `datetime.strptime(texte, "%d/%m/%Y").date()`. Cette fonction **lève une `ValueError`** si le texte ne correspond pas au format, **ou** si la date n'existe pas (31/02).
3. Refusez une date postérieure à `date.today()`.

🔎 **A chercher** : « Python datetime strptime » ; la documentation officielle : https://docs.python.org/fr/3/library/datetime.html (section *strftime() and strptime() Format Codes*).

💡 **A comprendre**

- Que signifient `%d`, `%m` et `%Y` ?
- Pourquoi n'avez-vous **pas** besoin d'écrire vous-même la vérification des années bissextiles ?
- Deux dates se comparent-elles avec `>` ? Que représente `date.today()` ?

✅ **Vérification** : `31/02/2026`, `2026-10-15`, `15/13/2026` et une date de l'année prochaine sont refusés ; `29/02/2024` et `15/10/2026` sont acceptés.

▼ Indice : le squelette de lire_date

```python
def lire_date(question):
    while True:
        texte = input(question).strip()
        try:
            jour = datetime.strptime(texte, "%d/%m/%Y").date()
        except ValueError:
            # message, puis tour suivant
            ...
        # tester le futur, puis renvoyer jour
```

```
git add .
git commit -m "Lit et vérifie la date de livraison"
```

Étape 6 : le quartier et les frais (30 min)

🎯 **Objectif** : le quartier est reconnu quelle que soit la casse (`plateau` , `PLATEAU`), et les frais sont calculés par une fonction qui lève des exceptions.

🛠️ **A faire**

1. `lire_quartier(question)` : comparez la saisie à chaque quartier de `TARIFS` **en minuscules**, et renvoyez le quartier **avec son orthographe officielle** (`"Plateau"`). Sinon, affichez la liste des quartiers possibles.
2. `frais_livraison(quartier, poids)` : lève une `ValueError` si le poids n'est pas positif, une `KeyError` si le quartier n'est pas desservi ; ajoute le supplément au-delà de `POIDS_LOURD` .
3. Testez `frais_livraison` **seule**, avec un petit bloc provisoire : `("Plateau", 2)` → 1500 ; `("Yoff", 7.5)` → 2500 ; `("Médina", -1)` et `("Pikine", 3)` dans un `try` qui affiche l'erreur attrapée.

💡 **A comprendre**

- `lire_quartier` garantit déjà un quartier valide : pourquoi `frais_livraison` vérifie-t-elle quand même ?
- Un colis de 5 kg exactement paie-t-il le supplément ? Vérifiez votre condition.

✅ **Vérification** : les quatre tests donnent 1500, 2500, puis les deux messages d'erreur. Retirez le bloc de test.

```
git add .
git commit -m "Lit le quartier et calcule les frais"
```

Étape 7 : saisir, ranger, récapituler (35 min)

🎯 **Objectif** : le programme complet de l'aperçu.

🛠️ A faire

1. `saisir_livraison()` : appelle les quatre fonctions de lecture et renvoie un **dictionnaire** avec les clés `date`, `quartier`, `poids`, `montant` et `frais`.
2. `afficher(livraison)` : une ligne alignée comme dans l'aperçu. Dans une f-string, `{livraison['date']:%d/%m/%Y}` formate une date, `{texte:<9}` aligne à gauche sur 9 caractères, `{nombre:>7}` à droite sur 7.
3. `main()` : lit le nombre de livraisons, les saisit dans une **liste**, affiche le récapitulatif et le total des frais.
4. En bas du fichier :

```
if __name__ == "__main__":
    main()
```

💡 A comprendre

- Pourquoi une liste de dictionnaires, plutôt que cinq listes séparées ?
- `if __name__ == "__main__":` sera expliqué au chapitre 6 : notez simplement que `main()` s'exécute quand on lance le fichier.

✅ **Vérification** : avec les saisies de l'aperçu (deux livraisons : Plateau 2,5 kg 12 500 F le 15/10/2026, et Yoff 7,5 kg 85 000 F le 16/10/2026), le récapitulatif est identique, et le total vaut 4000 F.

```
git add .
git commit -m "Saisit les livraisons et affiche le récapitulatif"
```

Étape 8 : résister à tout, puis étiqueter (20 min)

🎯 **Objectif** : même `Ctrl+C` ne produit plus de traceback.

🛠️ A faire

1. Lancez le programme et appuyez sur `Ctrl+C` pendant une saisie : quel traceback obtenez-vous ? Quel est le type de l'exception ?
2. Entourez l'appel de `main()` d'un `try` qui attrape `KeyboardInterrupt` **et** `EOFError` (fin de l'entrée, par exemple `Ctrl+Z` puis `Entrée` sous Windows) et affiche `Saisie interrompue.`.
3. Relisez tout le fichier : aucun `except:` seul, un `try` court dans chaque fonction, des messages qui disent quoi faire.
4. Enregistrez et étiquetez :

```
git add .
git commit -m "Gère l'interruption de la saisie"
git tag atelier-1
git push
git push origin atelier-1
```

💡 **A comprendre**

- D'après l'arbre de la section 6 du chapitre, pourquoi un `except Exception` n'attraperait-il **pas** le `Ctrl+C` ?
- Toutes les livraisons saisies sont perdues à la fermeture du programme. Que faudrait-il pour les garder ? (C'est le chapitre 2.)

✅ **Vérification** : `Ctrl+C` affiche `Saisie interrompue.` sans traceback ; sur GitHub, l'onglet **Tags** montre `atelier-1`.

🏆 Défis (facultatifs)

1. **Une exception à vous.** Créez `MontantInvalideError(ValueError)` et utilisez-la dans `verifier_montant` (chapitre 1, section 7). Le reste du programme doit fonctionner **sans modification** : pourquoi ?
2. **Corriger une livraison.** Après le récapitulatif, proposez de corriger une livraison par son numéro (redemandé s'il n'existe pas).
3. **Le téléphone du destinataire.** Ajoutez un numéro de 9 chiffres commençant par 7, espaces tolérés (`77 123 45 67`).
4. **Le bilan par quartier.** Affichez le nombre de livraisons et le total des frais par quartier (un dictionnaire de compteurs).

Grille d'autoévaluation

Je sais…	Oui	Pas encore
lire un traceback et trouver la ligne et le type de l'erreur	☐	☐
écrire un `try` / `except` qui vise la bonne exception	☐	☐
redemander une saisie tant qu'elle est invalide	☐	☐
lever une exception avec un message clair, et l'attraper ailleurs	☐	☐
convertir et vérifier une date avec `datetime.strptime`	☐	☐
expliquer pourquoi `except:` seul est dangereux	☐	☐
empêcher `Ctrl+C` de produire un traceback	☐	☐